



THM
TECHNISCHE HOCHSCHULE MITTELHESSEN

**CAMPUS
GIESSEN**

MNI

Mathematik, Naturwissenschaften
und Informatik



PE5001

Echtzeitsysteme

**Einführung in RTOS
und Prozessor**



Inhalte der Veranstaltung

Aus dem Modulhandbuch + Ergänzungen

**Echtzeit-
Betriebssysteme**

Tasks, Semaphore,
Message Queues

Exceptions und
Interrupts

Timer, I/O-Subsystem

Memory Management

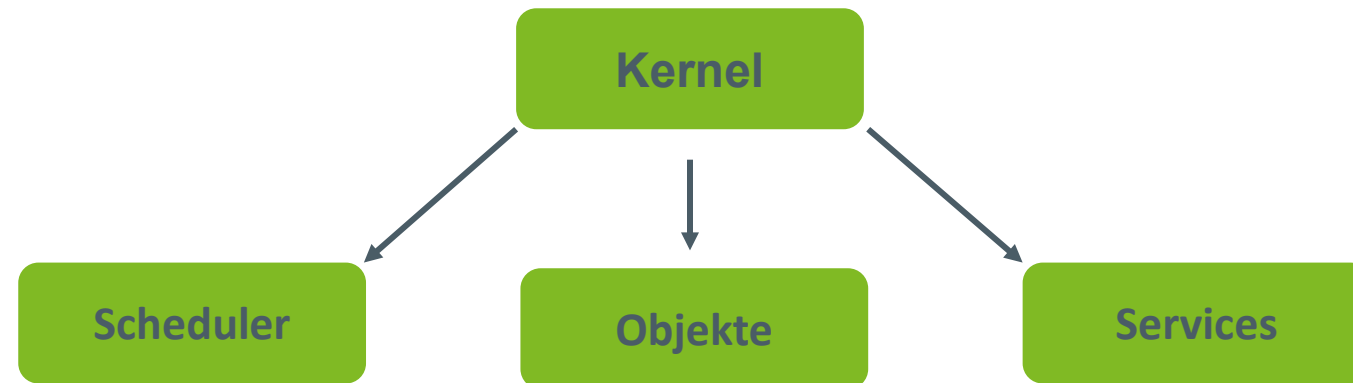
Rate-Monotonic
Scheduling

Synchronisation und
Kommunikation

Analysemethoden von
Echtzeit-anwendung

RTOS - Definition

- Ein *real-time operating system* (**RTOS**) ist ein Programm, welches...
 - ... Ausführungszeit zeitlich plant (Scheduler).
 - ... Systemressourcen managt.
 - ... eine Grundlage zur Entwicklung von Applikationen bildet.



RTOS

- Warum ein **RTOS**?
 - Zeitliche Restriktionen/Echtzeitanforderungen
 - Berechnungen müssen innerhalb zeitlicher Grenzen erfolgen/erfolgt sein (*Deadline*)



Kaffeemaschine



Rakete



Medizinische Geräte

Was ist Echtzeit?

- Echtzeit $\neq$ Performantes System
- Unter Echtzeit verstehen wir ein System, welches innerhalb einer definierten Zeit auf ein Ergebnis reagieren kann.
- Die Zeitspanne selbst wird dabei von dem übergeordneten System bestimmt.
 - Beispiele
 - Reaktionszeit **Crashsensor** für Airbag-Auslösung: 15-30ms¹
 - Reaktionszeit **Ampelsteuerungen** im Sekundenbereich

¹ <https://www.airbag24.de/blogs/ratgeber/wann-loest-crashsensor-aus>

RTOS

soft real-time



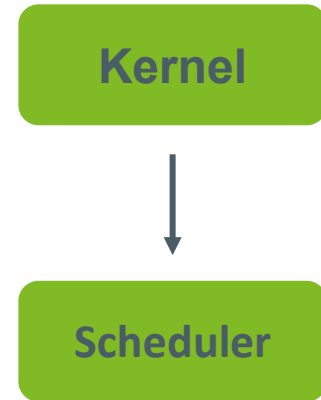
hard real-time

- Muss zeitliche Grenzen einhalten, jedoch mit einer gewissen Flexibilität.
- Durch Angabe einer durchschnittlichen zeitlichen Grenze.
- Eine nicht eingehaltene Deadline führt zu keinem Systemfehler. Aber Kosten können entstehen bzw. proportional zur Ausfallzeit steigen.
- Verfehlen der Zeitlichen Grenze (Deadline) beeinflusst nicht das Ergebnis. Ergebnis ist noch nützlich.

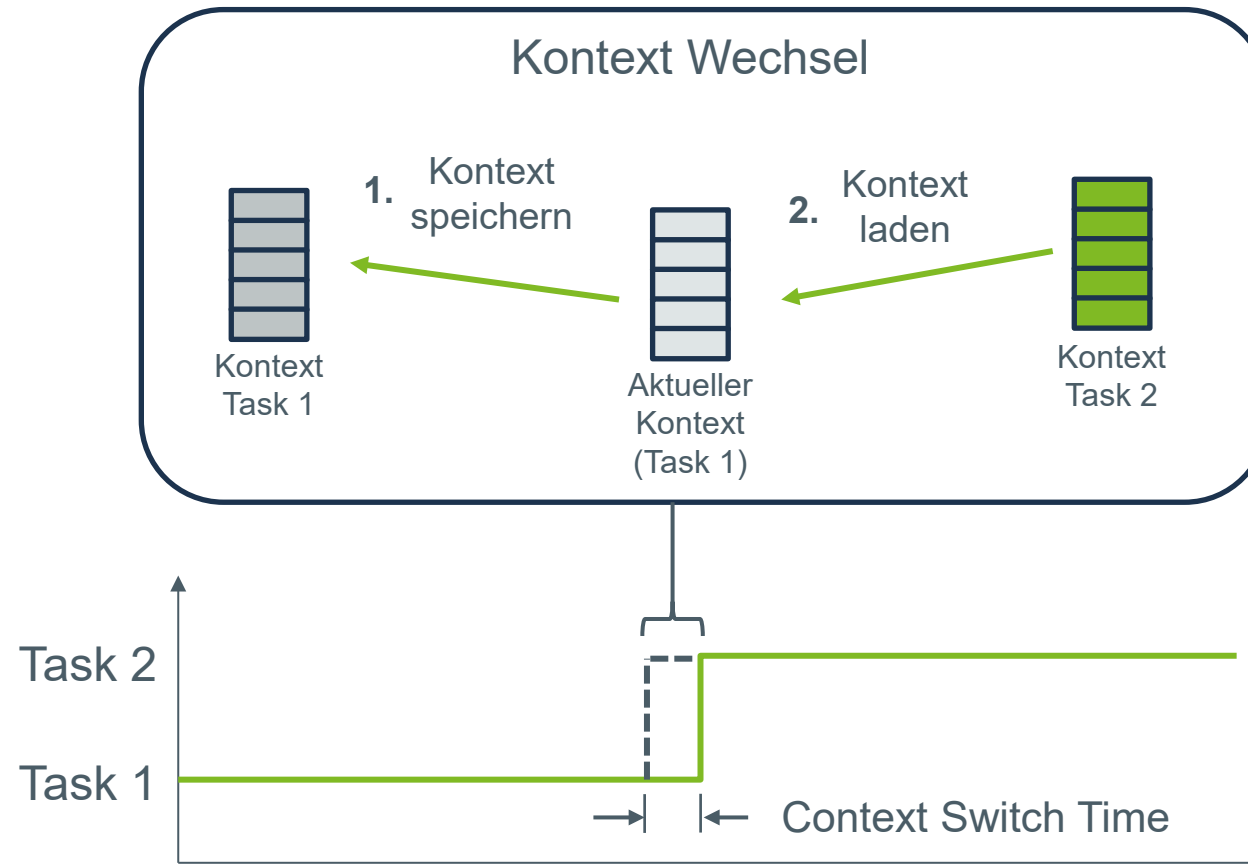
- Muss zeitliche Grenzen unbedingt einhalten – keine Flexibilität.
- Kann im Worst-Case Katastrophen auslösen oder Menschenleben gefährden.
- Kann zu einem Systemfehler führen.
- Ob ein System eine harte oder weiche Echtzeitanforderung besitzt, entscheidet letztendlich die Strafe, die bei nicht-einhaltung zu erwarten ist.

Der Scheduler

- Wir wollen Tasks/Prozesse quasi-parallel ausführen
 - Das nennt man *Multitasking* (im Kontext von Single-Cores)
 - Wird erreicht durch eine verschachtelte Ausführung der Tasks
- Der Scheduler ist das Herz des RTOS.
- Er muss mit seinem Scheduling-Algorithmus dafür sorgen, dass die Tasks bearbeitet werden und Ausführungszeit bekommen
- Das wird erreicht durch den Kontext-Wechsel (**Context Switch**). Das ist nichts anderes, als der Wechsel von einem Task in einen anderen.
- Beim **Context Switch** wird der *Zustand* des laufenden Tasks *gespeichert* und der *Zustand* des als nächsten auszuführenden Tasks *geladen*.



Der Kontext-Wechsel



Nach Abbildung 4.3 aus Real-Time
Concepts for Embedded Systems

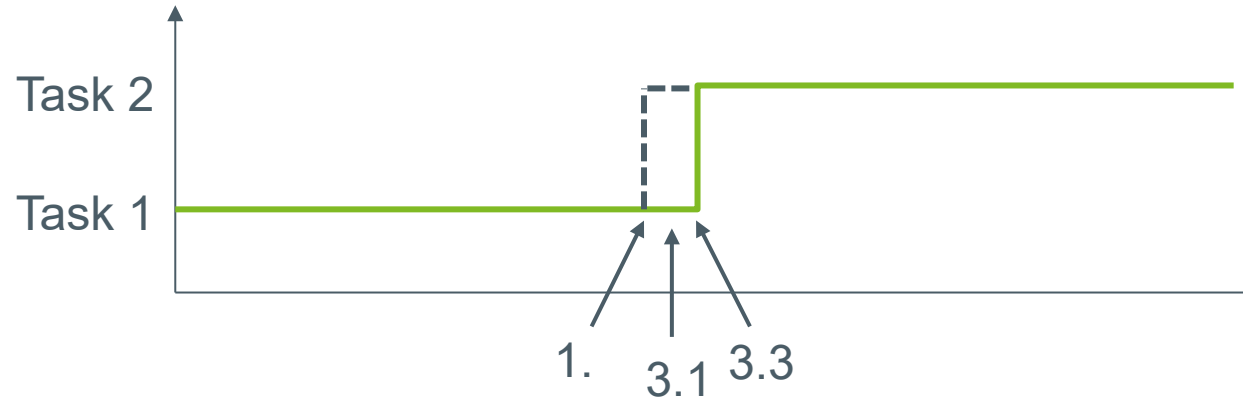
Der Kontext-Wechsel

- Was ist der Zustand eines Tasks?
 - Ganz einfach: Der aktuelle Zustand des Prozessors!
 - Also die CPU-Register, Stack-Pointer etc.
- Diese Informationen speichern wir in einem **Task-Control Block** (TCB) ab.
- Der Teil unseres RTOS, welches den Context Switch durchführt, nennt man **Dispatcher**
- Der **Dispatcher** ist also der Teil des Schedulers, welcher den Kontrollfluss unseres Programmes ändert
 - Indem durch den Context Switch ein anderer Task ausgeführt wird.

Der Dispatcher

- Der Scheduler wird normalerweise immer dann ausgeführt, wenn wir Systemfunktionen aufgerufen haben. Z.B. wenn:
 - ... wir ein **Mutex** wieder freigeben.
 - ... oder eine Nachricht in eine **Message Queue** geschrieben wurde.
- Dann entscheidet der Scheduler mit seinem Scheduling-Algorithmus, welcher Task als nächstes aufgeführt werden muss...
- ... und der **Dispatcher** übergibt durch den Context Switch die Kontrolle wohlmöglich an einen anderen Task.

Der Dispatcher



1. **Systemfunktion** (z.B. *Mutex unlock*) wird in **Task 1** aufgerufen.
2. Mutex wird freigegeben.
3. Scheduler wird aufgerufen.
 - 3.1 Scheduling Algorithmus wird ausgeführt.
 - 3.2 Entscheidung: Task 2 wird als nächstes ausgeführt.
 - 3.3 **Dispatcher** wird aufgerufen: Context Switch zu **Task 2**

Der Dispatcher

- Systemfunktionen können an unterschiedlichen Stellen im Code aufgerufen werden.
→ Je nachdem wie wir den Kernel betreten, kann das Dispatching unterschiedlich aussehen

Task → Kernel (durch Systemfunktion)

- Dispatcher wird am **Ende** der Systemfunktion aufgerufen
- Task kann durch Aufruf der Systemfunktion unterbrochen werden

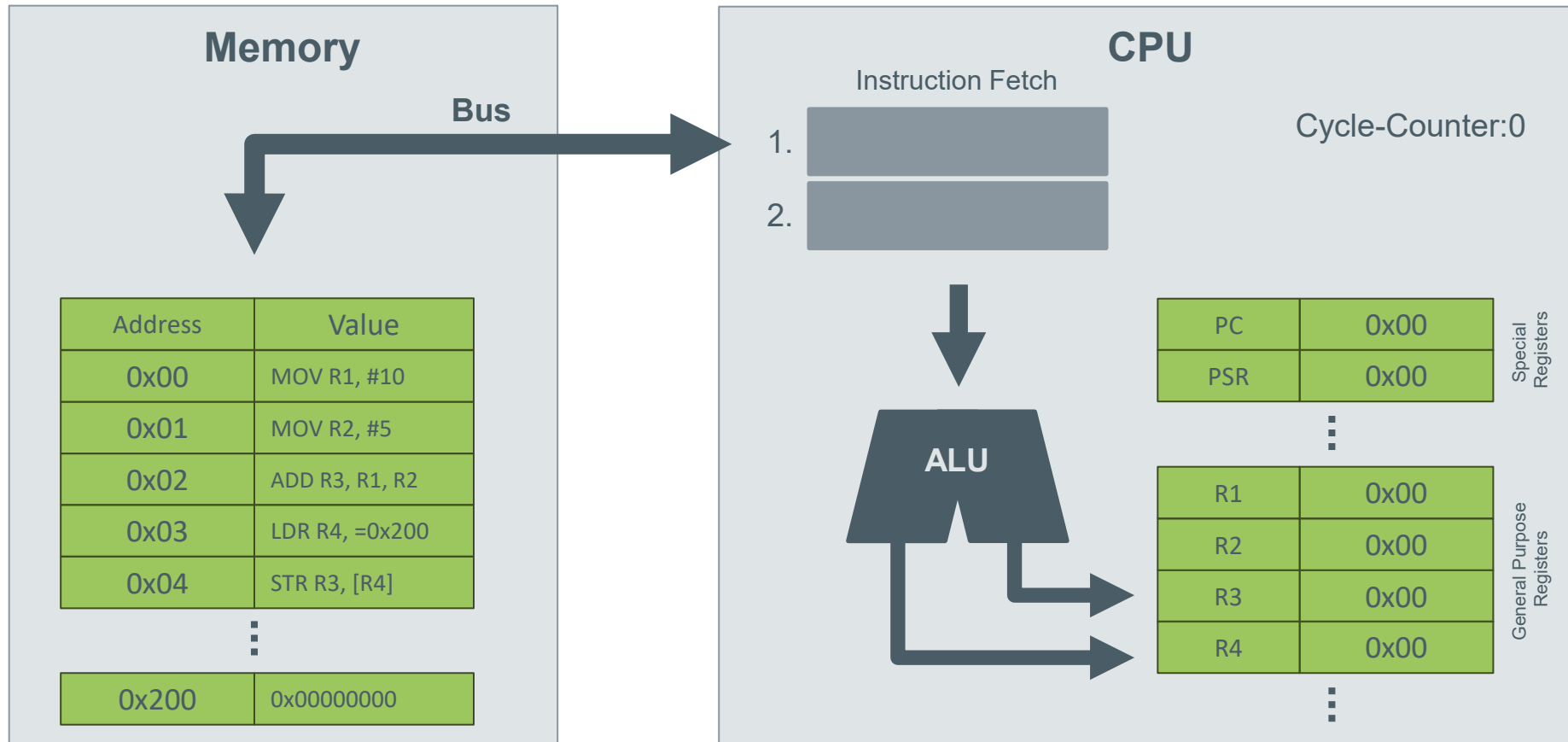
ISR → Kernel (durch Systemfunktion)

- Das Dispatching kann nicht am Ende der Systemfunktion aufgerufen werden. Die ISR muss beendet werden können!
- Am Ende der ISR wird stattdessen in den Kernel gewechselt, wobei dann der Dispatcher aufgerufen wird.

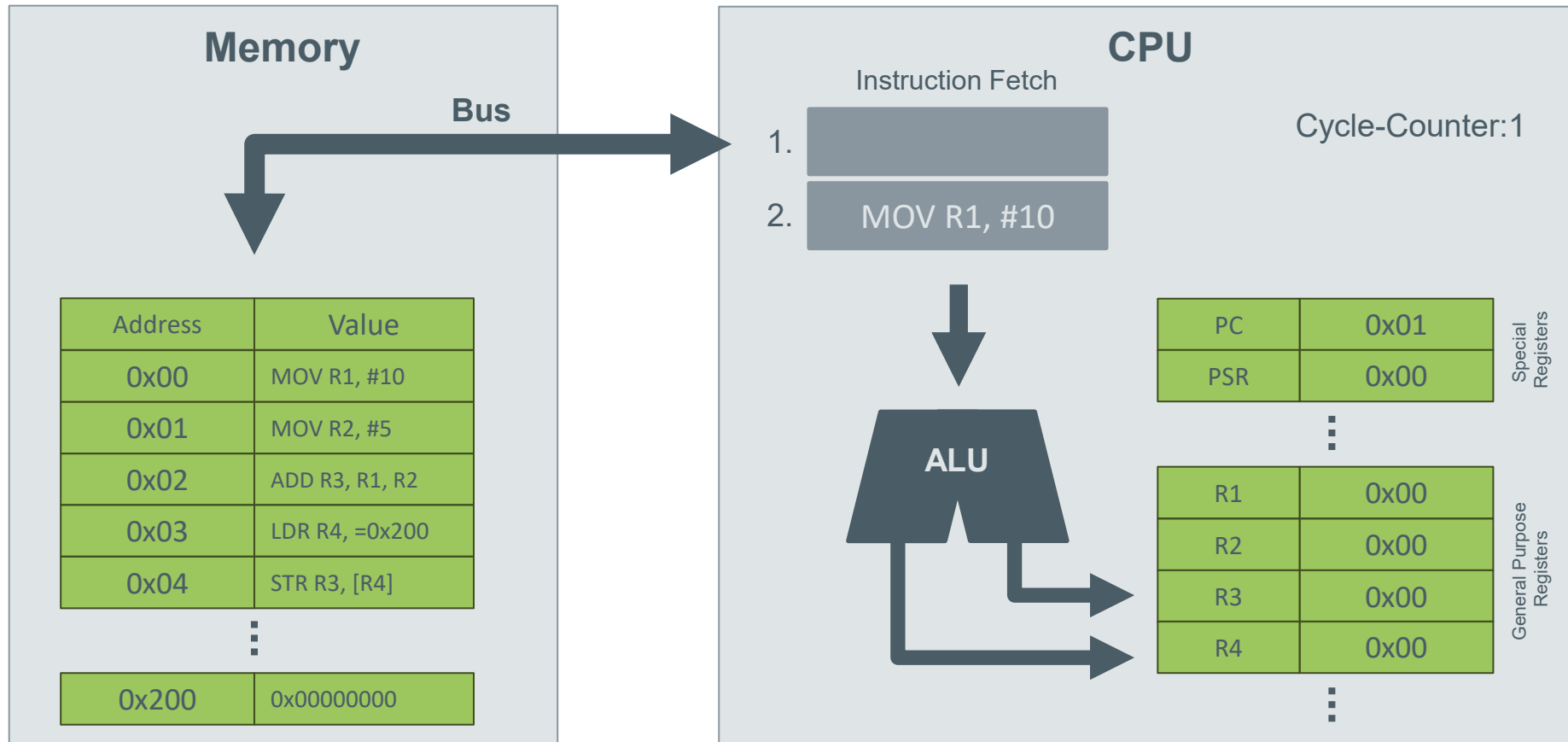
Der Dispatcher

- Gibt es da nicht noch etwas in Prozessoren, wo auch der Zustand der CPU gespeichert werden muss?
 - Bei Interrupts!
 - Es ist also extrem wahrscheinlich, dass es extra Instruktionen, Handler oder generell Hilfsmittel gibt, die uns das Speichern des Prozessorzustandes ermöglichen

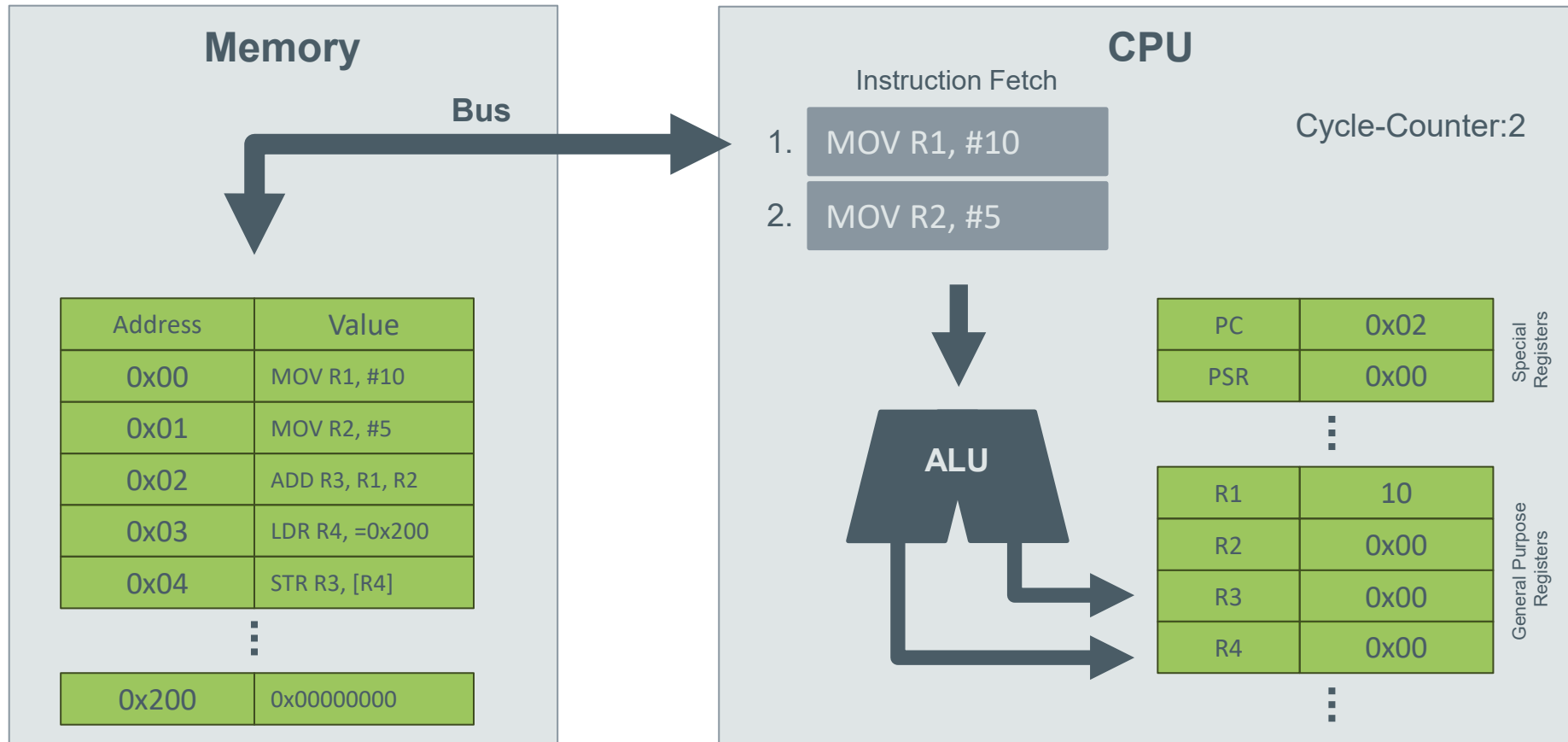
Wie funktioniert nochmal eine CPU? (Vereinfacht, ohne FPU)



Wie funktioniert nochmal eine CPU? (Vereinfacht, ohne FPU)

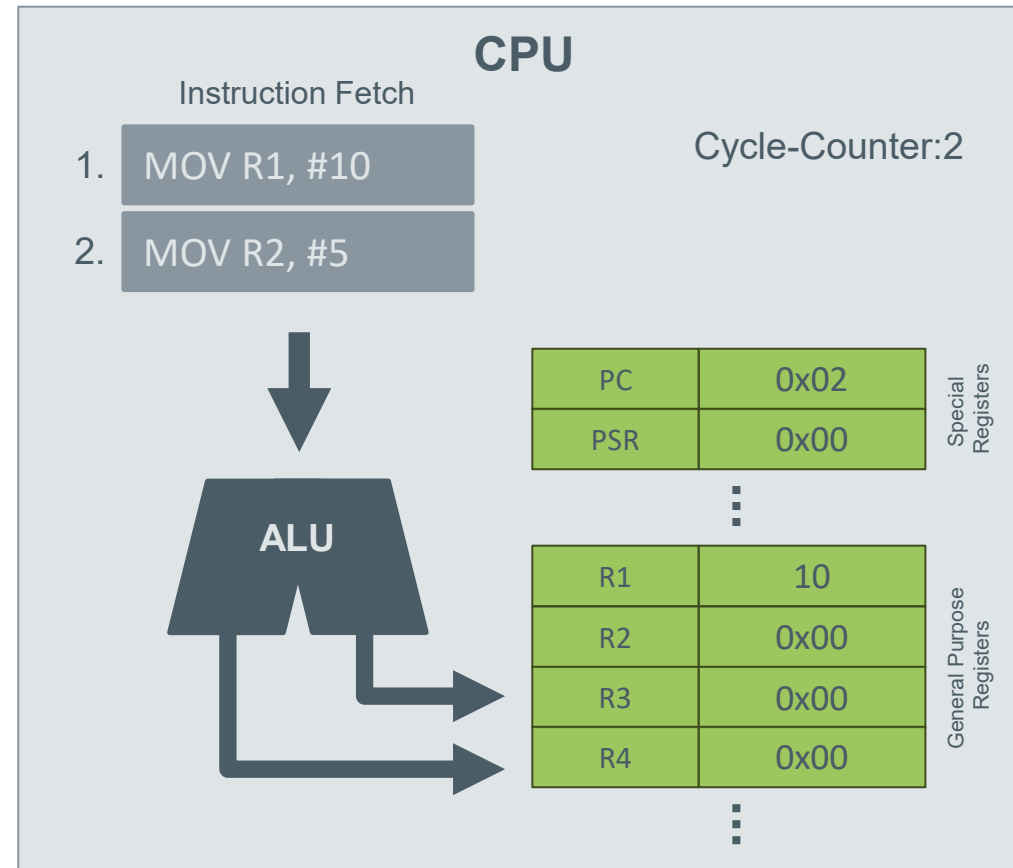


Wie funktioniert nochmal eine CPU? (Vereinfacht, ohne FPU)

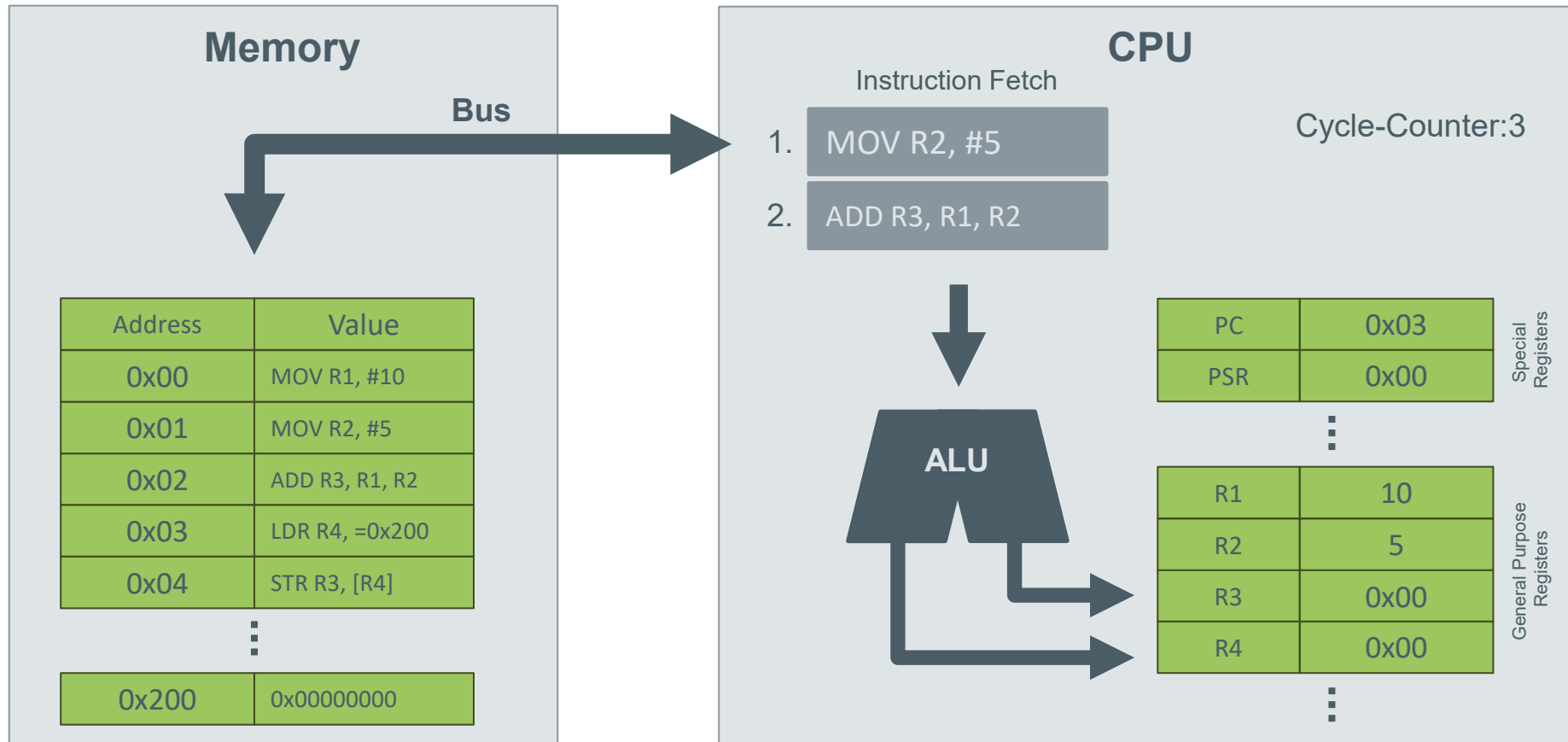


Wie funktioniert nochmal eine CPU? (Vereinfacht, ohne FPU)

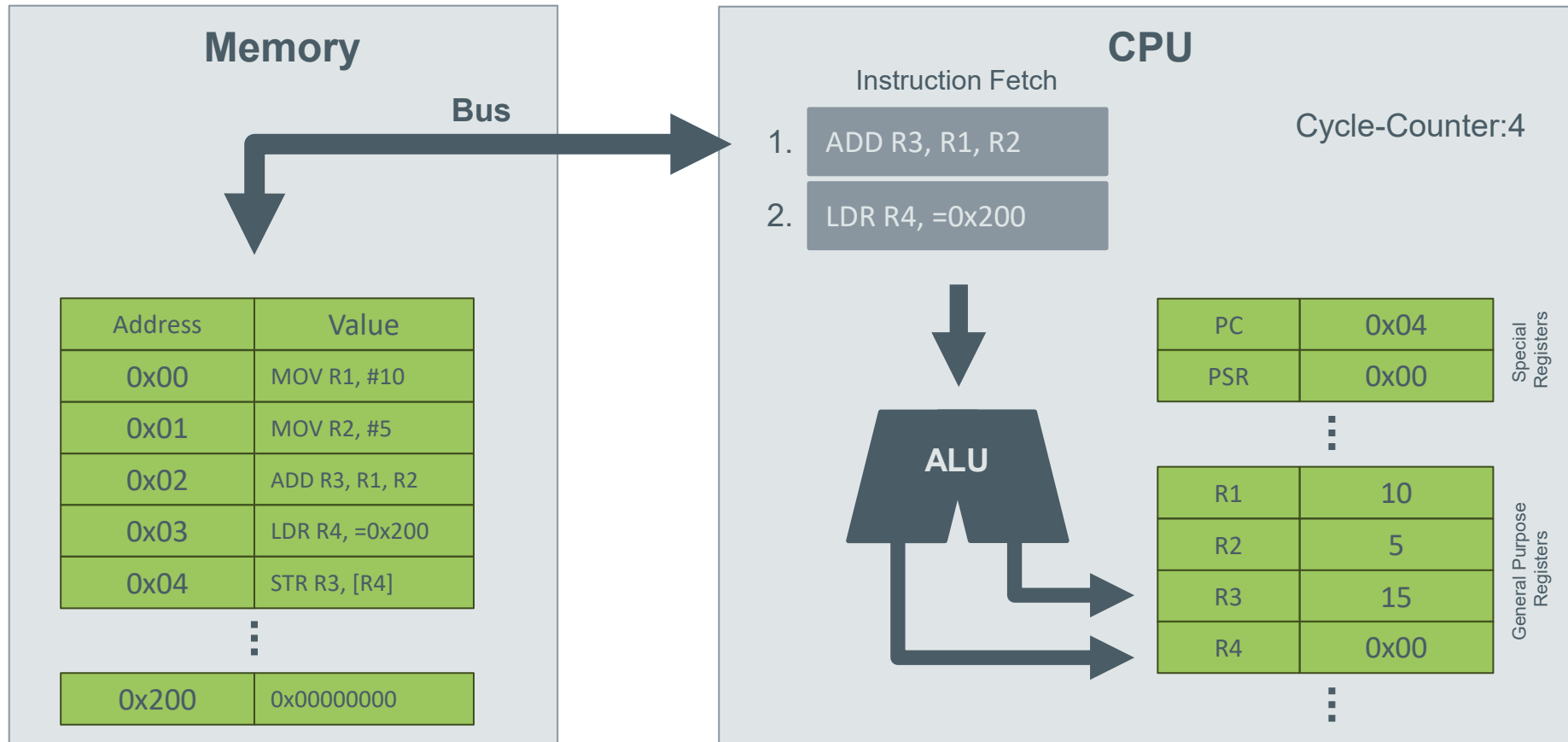
Was ist der aktuelle Zustand der CPU?



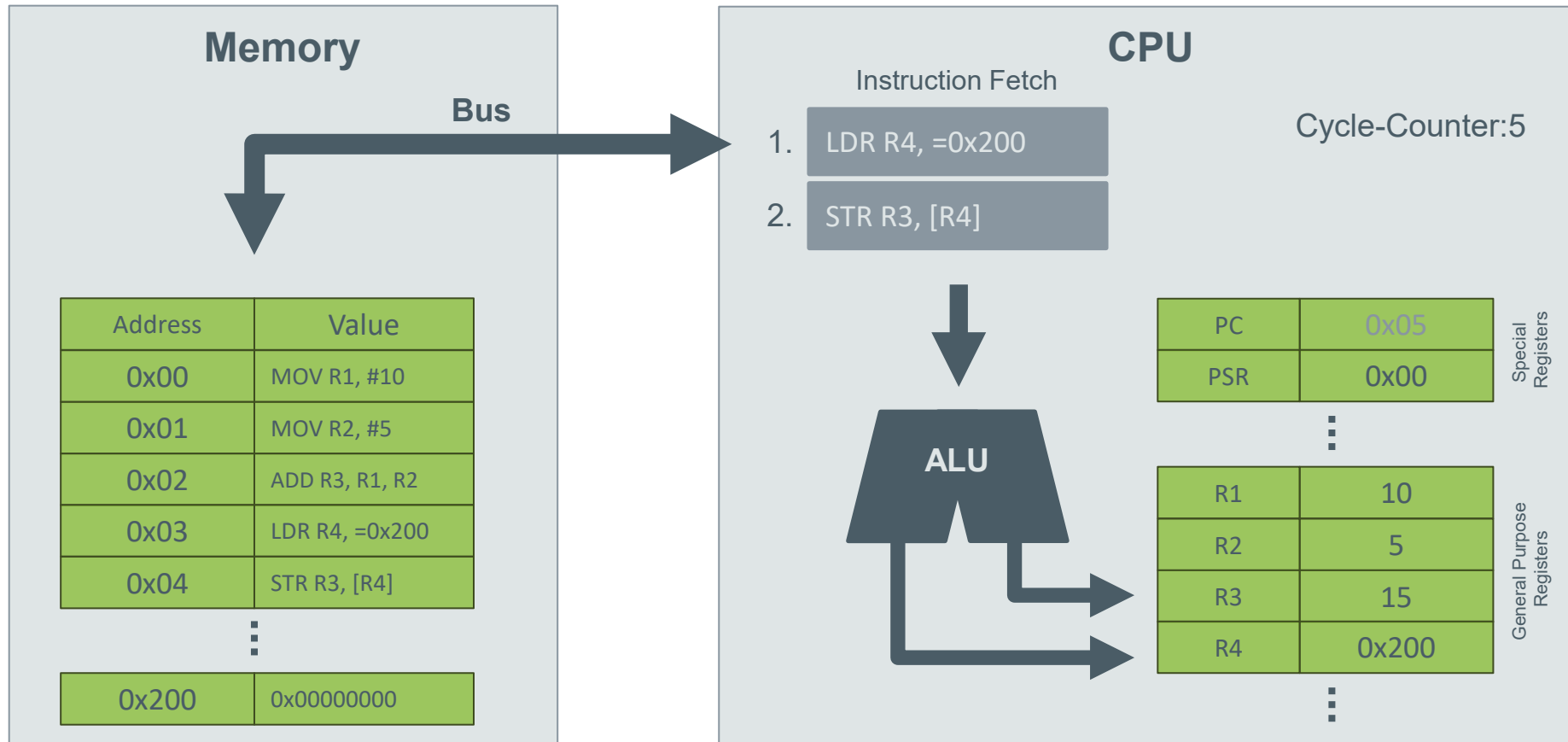
Wie funktioniert nochmal eine CPU? (Vereinfacht, ohne FPU)



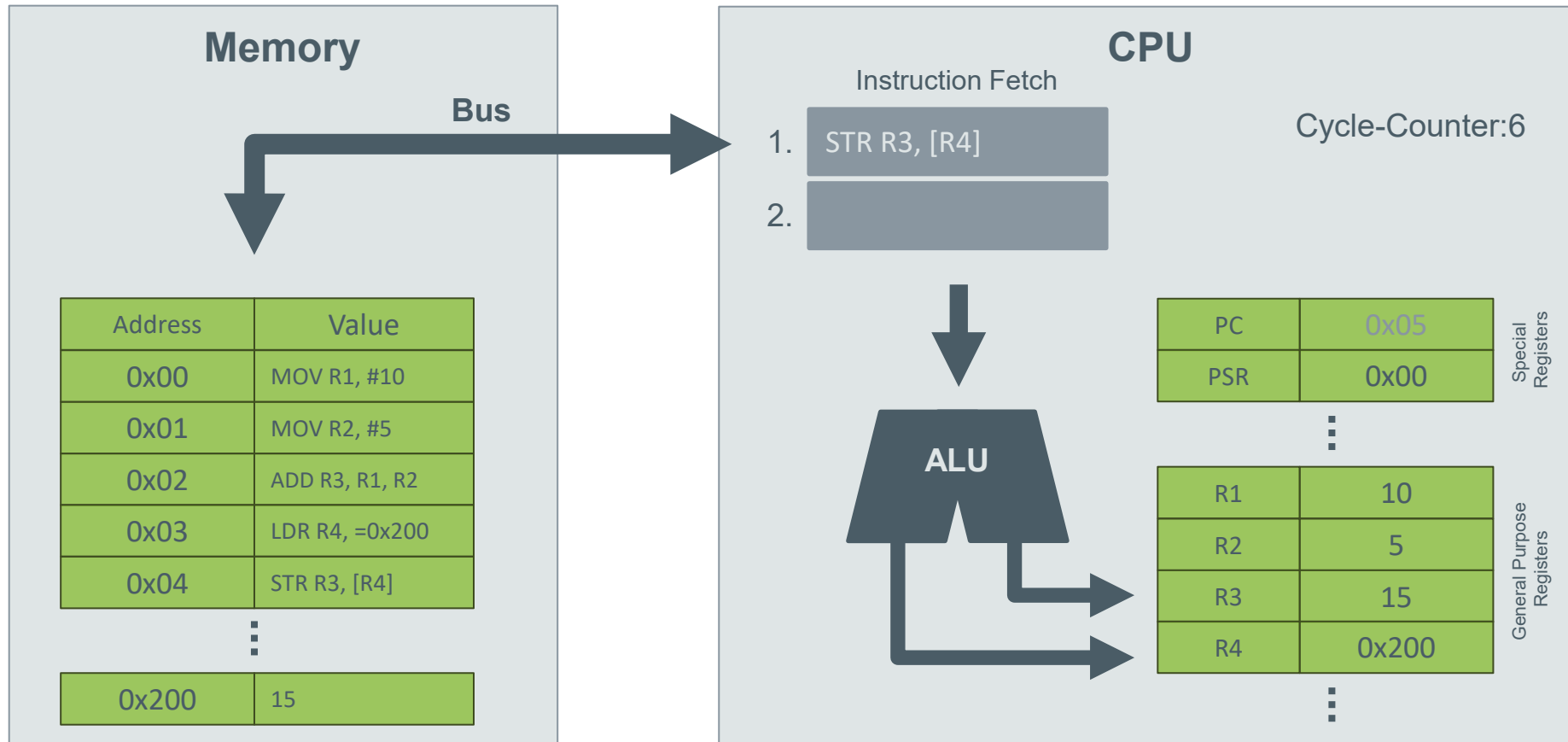
Wie funktioniert nochmal eine CPU? (Vereinfacht, ohne FPU)



Wie funktioniert nochmal eine CPU? (Vereinfacht, ohne FPU)



Wie funktioniert nochmal eine CPU? (Vereinfacht, ohne FPU)



Cortex M4 Program manual:

2.1.3 Core registers

Figure 2. Processor core registers

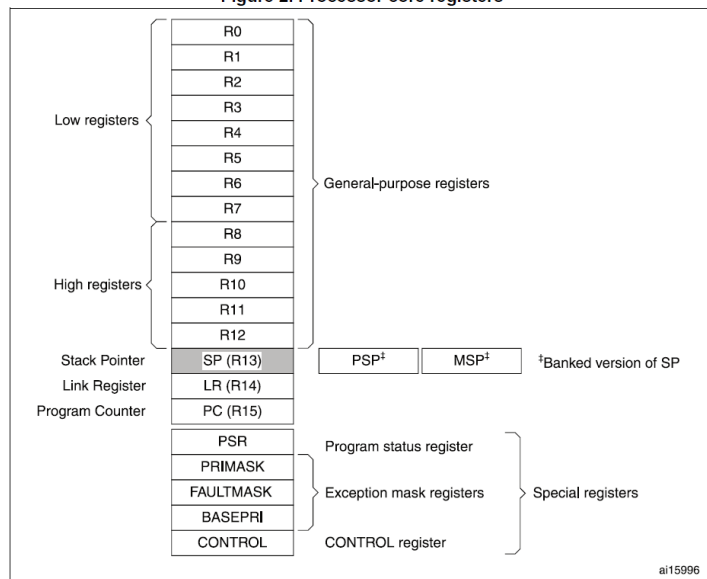


Table 3. Core register set summary

Name	Type ⁽¹⁾	Required privilege ⁽²⁾	Reset value	Description
R0-R12	read-write	Either	Unknown	General-purpose registers on page 19
MSP	read-write	Privileged	See description	Stack pointer on page 19
PSP	read-write	Either	Unknown	Stack pointer on page 19
LR	read-write	Either	0xFFFFFFFF	Link register on page 19
PC	read-write	Either	See description	Program counter on page 19
PSR	read-write	Privileged	0x01000000	Program status register on page 19
ASPR	read-write	Either	Unknown	Application program status register on page 21
IPSR	read-only	Privileged	0x00000000	Interrupt program status register on page 22
EPSR	read-only	Privileged	0x01000000	Execution program status register on page 22
PRIMASK	read-write	Privileged	0x00000000	Priority mask register on page 24
FAULTMASK	read-write	Privileged	0x00000000	Fault mask register on page 24
BASEPRI	read-write	Privileged	0x00000000	Base priority mask register on page 25
CONTROL	read-write	Privileged	0x00000000	CONTROL register on page 25

1. Describes access type during program execution in thread mode and Handler mode. Debug access can differ.

2. An entry of either means privileged and unprivileged software can access the register.

Was war nochmal das Link Register?

Und der Stack Pointer?

Dispatching auf dem Cortex M4

- Supervisor Call wird also für bestimmte Instruktionen benötigt.
- Mit dem PendSV können wir einen Request auslösen, welcher eine Exception (und damit einen Exception handler aufruft), der für unseren Context-Switch verwendet werden kann.
- Stacks sind absteigend implementiert. Die Adresse des Stack-Pointers wird dekrementiert.

The *privilege levels* for software execution are:

Unprivileged: *Unprivileged software* executes at the unprivileged level and:

- Has limited access to the MSR and MRS instructions, and cannot use the CPS instruction.
- Cannot access the system timer, NVIC, or system control block.
- Might have restricted access to memory or peripherals.
- Must use the SVC instruction to make a supervisor call to transfer control to privileged software.

Privileged: *Privileged software* executes at the privileged level and can use all the instructions and has access to all resources.
Can write to the CONTROL register to change the privilege level for software execution.

SVC	<i>A supervisor call (SVC) is an exception that is triggered by the SVC instruction. In an OS environment, applications can use SVC instructions to access OS kernel functions and device drivers.</i>
PendSV	<i>PendSV is an interrupt-driven request for system-level service. In an OS environment, use PendSV for context switching when no other exception is active.</i>

2.1.2 Stacks

The processor uses a full descending stack. This means the stack pointer indicates the last stacked item on the stack memory. When the processor pushes a new item onto the stack, it decrements the stack pointer and then writes the item to the new memory location. The processor implements two stacks, the *main stack* and the *process stack*, with independent copies of the stack pointer, see [Stack pointer on page 19](#).

Dispatching auf dem Cortex M4

- Stack Frame: Beim Eintritt in eine Exception (also auch, wenn wir PendSV triggern)!
- Beim Stack-Frame sind aber nicht alle Register enthalten! Also müssen wir manuell noch nachhelfen. Unsere General-Purpose Register gehen bis R12, R4-R11 fehlen hier.
- Der Stack-Pointer auch

2.1.3 Core registers

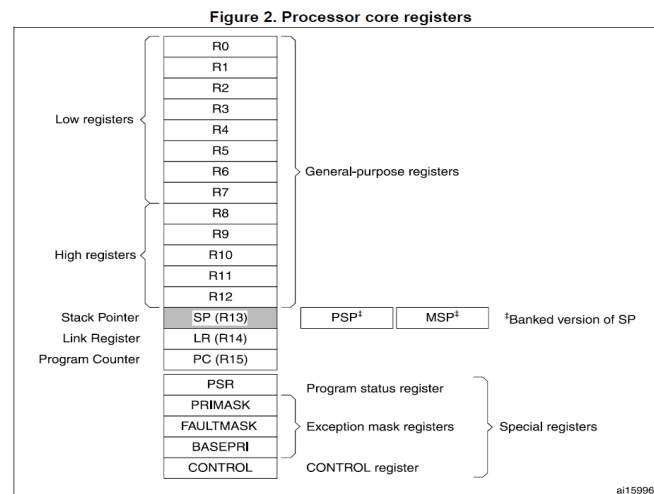
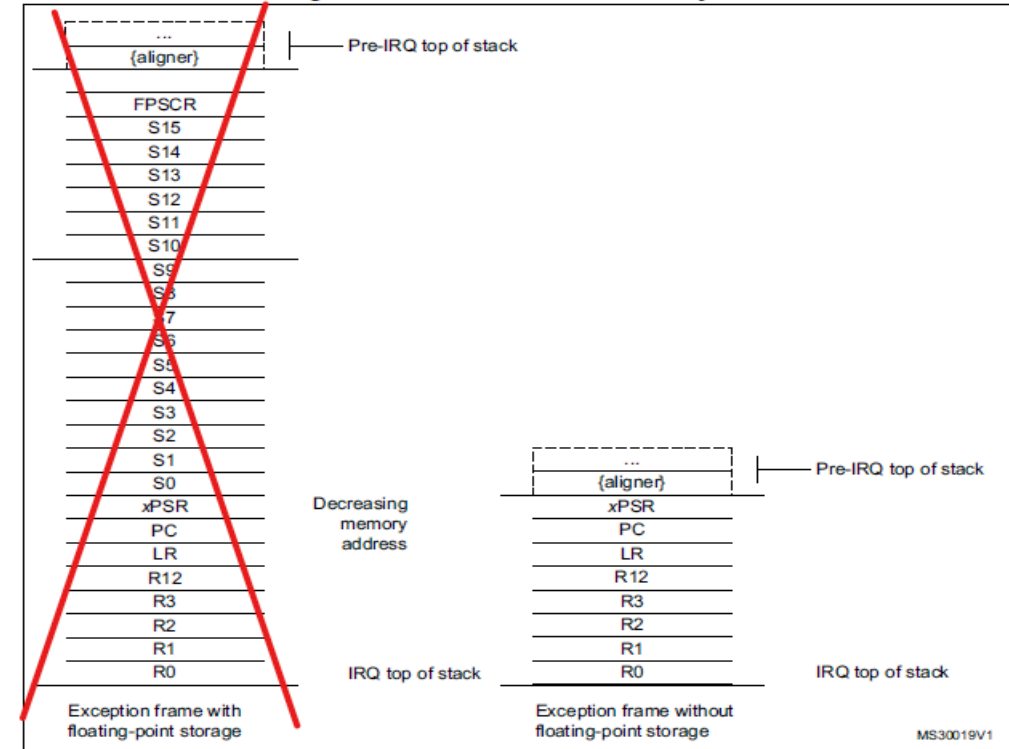


Figure 12. Cortex-M4 stack frame layout



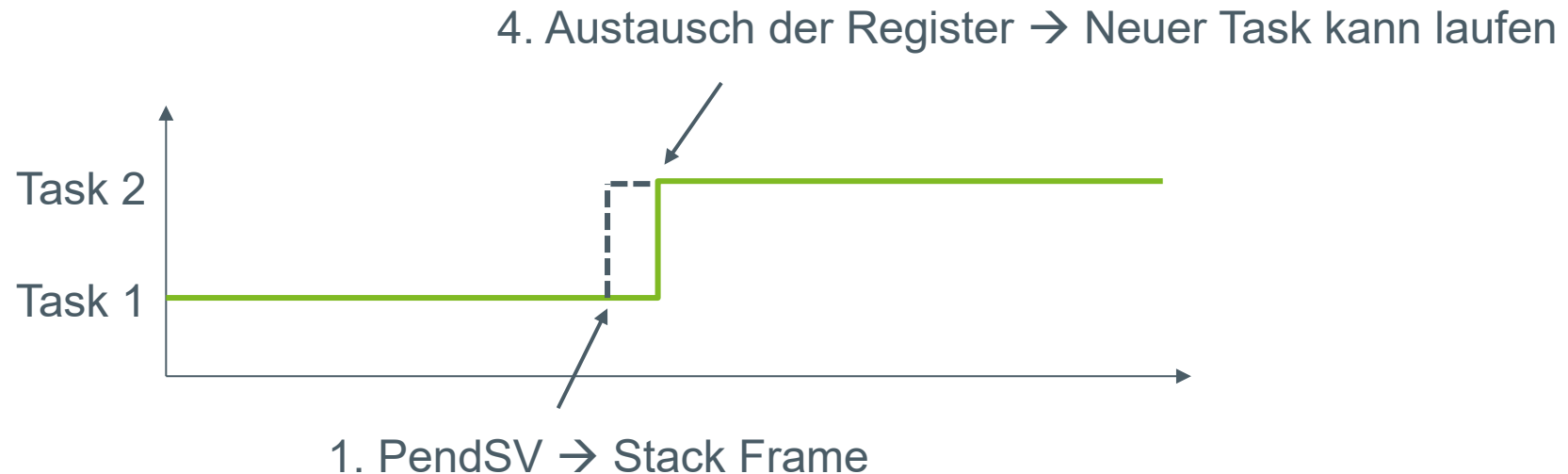
Immediately after stacking, the stack pointer indicates the lowest address in the stack frame. The alignment of the stack frame is controlled via the STKALIGN bit of the Configuration Control Register (CCR).

The stack frame includes the return address. This is the address of the next instruction in the interrupted program. This value is restored to the PC at exception return so that the interrupted program resumes.

In parallel to the stacking operation, the processor performs a vector fetch that reads the exception handler start address from the vector table. When stacking is complete, the processor starts executing the exception handler. At the same time, the processor writes an EXC_RETURN value to the LR. This indicates which stack pointer corresponds to the stack frame and what operation mode the processor was in before the entry occurred.

Dispatching auf dem Cortex M4

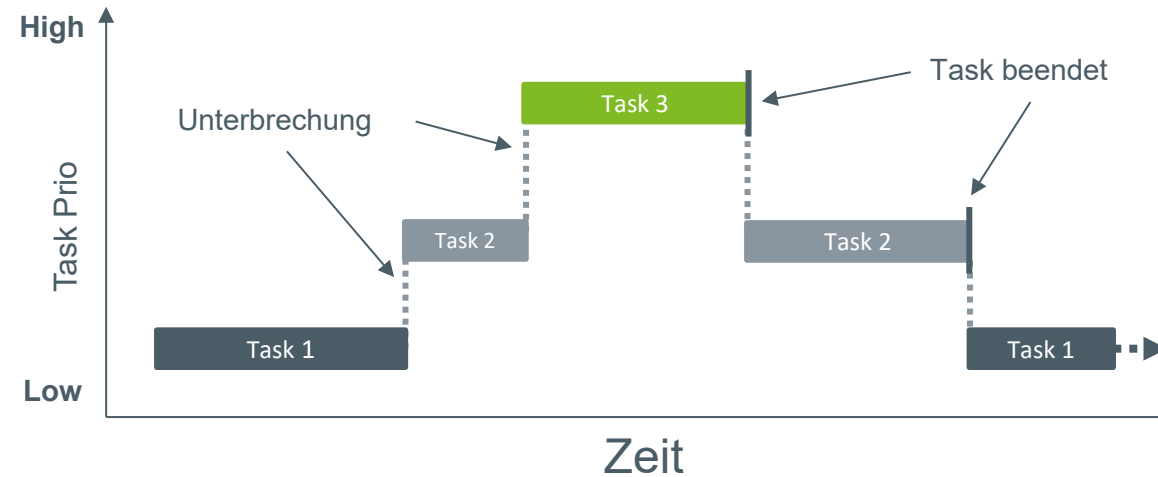
1. PendSV pusht unseren Stack-Frame auf den Stack (ganz automatisch).
2. Interrupts ausschalten
3. Push R4-R11 (restlichen Register) auf den Stack
4. Laden des neuen Tasks = Stack von Task 2, also alle Register der CPU austauschen in den Zustand/Stack von Task 2
5. Interrupts einschalten
6. PendSV Routine verlassen (Stack-Frame wird vom Stack genommen)



Scheduling Algorithmen

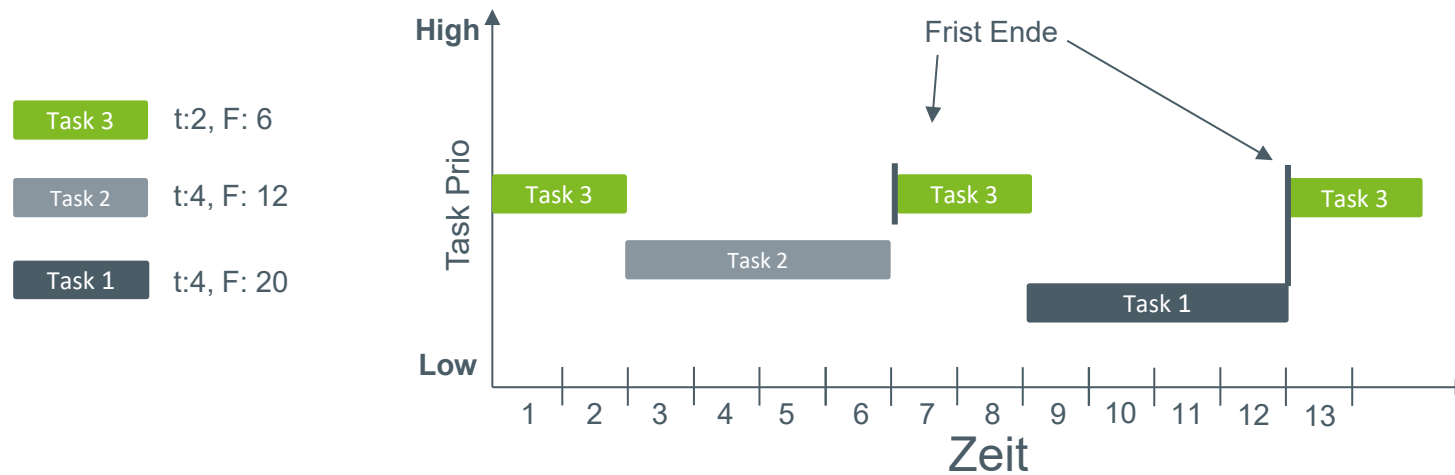
- Der Scheduling Algorithmus sorgt für die Auswahl des Tasks, der als nächstes ausgeführt werden soll.
- Man unterscheidet zwischen Präemptiven und nicht-präemptiven Verfahren
 - Round-Robin: Feste Zeitscheiben/Laufzeiten für Tasks
 - First-In First-Out
 - Shortest-Job-Next
 - Prioritätsbasierte Verfahren:
 - Präemptives prioritätsbasiertes Scheduling Verfahren
 - Rate-Monotonic Scheduling Verfahren
 - ...
- Einige Verfahren fallen bei unseren Echtzeitanforderungen raus!

Scheduling Algorithmen – Präemptives prioritätsbasiertes Scheduling



- Prioritätsvergabe kann Dynamisch oder Periodisch erfolgen
- Echtzeitsysteme mit weichen & harten Echtzeitanforderungen
- Beispiele:
 - Earliest Deadline First
 - Fixed Priority Scheduling
 - Rate-Monotonic Scheduling

Scheduling Algorithmen – Spezieller Fall: Rate-Monotonic Scheduling



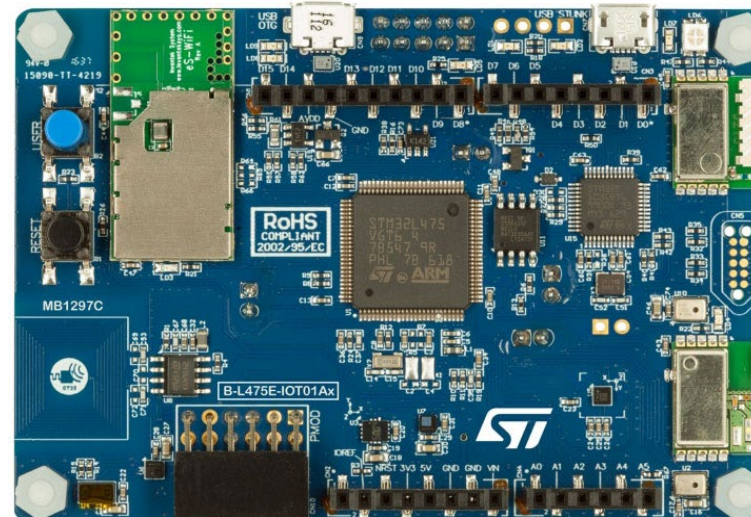
- Rate-Monotonic Scheduling (RMS) Verfahren
 - Die (statische) Priorität wird auf Basis der kürzesten Jobdauer festgelegt.
 - Je kürzer die Jobdauer, desto höher die Priorität.
 - Alle Tasks sind periodisch
 - Optimal für harte Echtzeitanforderungen und periodische Tasks.

Zusammenfassung

- **Echtzeitsysteme** sollen eine Aufgabe/Berechnung innerhalb einer bestimmten *Deadline* ausführen/berechnen.
- Ein **RTOS** besitzt einen Kernel und...
 - ... plant die Ausführungszeit von Tasks (Scheduling)
 - ... managt Systemressourcen (Objekte) wie Mutexe, Message Queues, ...
 - ... stellt Services bereit wie Interrupt Service Routinen
- Der **Scheduler** führt Tasks quasi-parallel durch Multitasking aus
- Beim **Dispatching** wird ein *Context-Switch* vollzogen.
 - Der Zustand (kann) von einem Task in einen anderen Task wechseln
 - Das Dispatching unterscheidet sich je nach Systemfunktionsaufruf Task/ISR
- **Scheduling-Algorithmen** sorgen für die Auswahl des nächsten Tasks
 - Algorithmen können *Präemptiv* (unterbrechend) oder *nicht-präemptiv* sein
 - Präemptives prioritätsbasiertes- oder rate-monotonic Scheduling eignen sich u.a. für Echtzeitsysteme: Tasks bekommen eine Priorität und werden teils periodisch ausgeführt bzw. unterbrochen.

First-Try

- Ladet euch die IDE runter
- Richtet das Board ein
- Erstellt ein erstes minimales Basisprojekt (führe ich vor)



The board's serial number is indicated on a sticker under the MB1297 reference on the bottom of the board. If this number is lower than 182404896 for B-L475E-IOT01A1C boards, or than 184906074 for B-L475E-IOT01A2C boards, the default firmware connecting to AWS Cloud needs updating. Download the latest version available at <http://www.st.com/x-cube-aws>